

◆ What is a String?

A **string** is a collection (sequence) of characters stored in a **character array**.

In C, every string **must end with a null character '\0'**, which tells the compiler where the string ends.

✓ Why '\0' is Important?

- It marks the **end of the string**
- Without it, the program may read **garbage values**
- All string functions rely on '\0'

Example:

```
char name[] = "Vansh";
```

Memory Representation:

Index: 0 1 2 3 4

Value: V a n s \0

👉 Even though "Vansh" has 4 characters, it occupies **5 memory locations**.

◆ Strings as Character Arrays

In C, strings are implemented using **character arrays (char[])**.

Key Points:

- Stored as an array of char
- Last element is always '\0'
- Size must be **(number of characters + 1)**

Example:

```
char str[10] = {'H','e','l','l','o','\0'};
```

Valid Alternate Way:

```
char str[10] = "Hello";
```

✗ Common Mistake:

```
char str[5] = "Hello"; // ✗ No space for '\0'
```

◆ Built-in String Functions (<string.h>)

C provides several **predefined functions** to work with strings.

To use them, include:

```
#include <string.h>
```

1 strlen() – Find Length of String

◆ Purpose:

Returns the **number of characters in a string**, excluding the null character '\0'.

Syntax:

```
strlen(string_name);
```

Example:

```
strlen("Hello"); // Output: 5
```

Internal Working:

- Starts counting from index 0
- Stops when '\0' is encountered
- Does **not count** '\0'

Code Example:

```
char str[] = "Computer";  
printf("%d", strlen(str)); // Output: 8
```

✦ **Time Complexity:** O(n)

2 strcpy() – Copy One String into Another

◆ Purpose:

Copies the contents of one string (**source**) into another string (**destination**).

Syntax:

```
strcpy(destination, source);
```

Example:

```
char src[] = "C Language";  
char dest[20];
```

```
strcpy(dest, src);  
printf("%s", dest);
```

Output:

C Language

Important Points:

- ✓ Destination array must be large enough
 - ✓ Copies characters **including** '\0'
 - ✗ Overlapping strings cause undefined behavior
-

3 strcmp() – Compare Two Strings

◆ Purpose:

Compares two strings **lexicographically (ASCII based)**.

Syntax:

```
strcmp(str1, str2);
```

Return Values:

Return Value Meaning

- | | |
|-----|---------------------------|
| 0 | Both strings are equal |
| < 0 | str1 is smaller than str2 |
| > 0 | str1 is greater than str2 |

Example:

```
strcmp("apple", "apple"); // 0
```

```
strcmp("apple", "banana"); // < 0
```

```
strcmp("cat", "car"); // > 0
```

Internal Working:

- Compares character by character
- Stops at first mismatch or '\0'
- Comparison is based on **ASCII values**

✚ 'a' < 'b' < 'c'

4 strcat() – Concatenate Strings

◆ Purpose:

Appends (joins) the **second string** to the **end of the first string**.

Syntax:

```
strcat(str1, str2);
```

Example:

```
char str1[20] = "Hello ";
```

```
char str2[] = "World";
```

```
strcat(str1, str2);
```

```
printf("%s", str1);
```

Output:

Hello World

Important Points:

- ✓ str1 must have enough space
 - ✓ str2 remains unchanged
 - ✓ '\0' of str1 is replaced during concatenation
-

 Summary Table**Function Work**

strlen() Finds length

strcpy() Copies string

strcmp() Compares strings

strcat() Joins strings

 Exam & Interview Tips

- Always mention **null character**
- Avoid using gets() (unsafe)
- Prefer fgets() for input
- Understand **manual implementation** of string functions

 Practice Problems (Concept + Logic)

 Reverse String**Idea:**

- Swap characters from start & end
- Move pointers inward

```
for(i = 0, j = len-1; i < j; i++, j--)
```

```
swap(str[i], str[j]);
```

2 Remove Outermost Parentheses

Idea:

- Track opening & closing brackets
- Skip first (and last)

Used in balanced parentheses problems.

3 Largest Odd Number in String

Idea:

- Traverse from right
 - First digit which is **odd**
 - Return substring till that index
-

4 Length of Last Word

Idea:

- Start from end
 - Skip spaces
 - Count characters until space
-

5 Length of String (Without strlen)

Logic:

```
count = 0;
while(str[count] != '\0')
    count++;
```

6 Count Number of Consistent Strings

Idea:

- Each character must belong to **allowed string**
 - Use frequency array or find()
-

Longest Common Prefix

Idea:

- Compare characters index-wise
 - Stop at first mismatch
-

Rotate String

Idea:

- Rotate left or right
- Use substring + concatenation

Example:

abcde → cdeab (left rotate by 2)

Activity (Important)

- ◆ Copy One String to Another WITHOUT strcpy()

 Objective:

Create a program to copy one string into another **manually**

◆ Logic:

- Traverse source string
 - Copy each character
 - End with '\0'
-

Program (C)

```
#include <stdio.h>
```

```
int main() {
```

```
    char src[100], dest[100];
```

```
    int i = 0;
```

```
    printf("Enter source string: ");
```

```
gets(src);

while (src[i] != '\0') {
    dest[i] = src[i];
    i++;
}

dest[i] = '\0'; // null terminate

printf("Copied string: %s", dest);

return 0;
}
```

Dry Run Example:

```
src = "Hello"
```

```
dest = ""
```

Iteration:

```
dest[0] = 'H'
```

```
dest[1] = 'e'
```

```
dest[2] = 'l'
```

```
dest[3] = 'l'
```

```
dest[4] = 'o'
```

```
dest[5] = '\0'
```